

Agile Methodologies

CMP9134: Software Engineering

Dr Francesco Del Duchetto
Lecturer in Robotics and Autonomous Systems

University of Lincoln

13 February 2026

Today's Agenda

- 1 Agile Overview
- 2 Extreme Programming (XP)
- 3 Scrum
- 4 Agile for Solo Projects
- 5 Next Steps

Agenda

- 1 Agile Overview
- 2 Extreme Programming (XP)
- 3 Scrum
- 4 Agile for Solo Projects
- 5 Next Steps

Agile Software Development

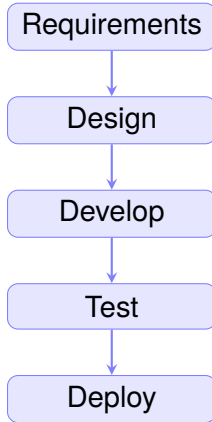
Definition

A set of methods and practices based on the values and principles expressed in the **Agile Manifesto**.

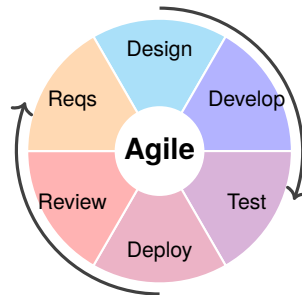
- Emphasises flexibility, continuous improvement, and high-quality results.
- Delivered in a collaborative environment with frequent stakeholder communication.
- **Popular Methodologies:** Scrum, Kanban, Extreme Programming (XP), Lean Development.

Agile vs Waterfall

Waterfall



Agile



The Agile Manifesto

Created in February 2001 at Snowbird, Utah, by 17 software developers looking for an alternative to documentation-driven, heavyweight software development processes.

"We are uncovering better ways of developing software by doing it and helping others do it."

■ Manifesto: <https://agilemanifesto.org/>

The Four Core Values

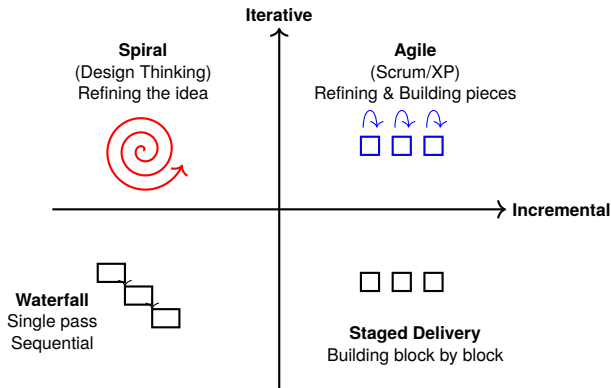
- 1 **Individuals and interactions** over processes and tools.
Collaboration > bureaucracy.
- 2 **Working software** over comprehensive documentation.
Value delivered > paper trails.
- 3 **Customer collaboration** over contract negotiation.
Cooperation > legalities.
- 4 **Responding to change** over following a plan.
Adaptability > rigidity.

That is, while there is value in the items on the right, we value the items on the left more.

The Twelve Principles (Summary)

- 1 Customer satisfaction through early/continuous delivery.
- 2 Welcome changing requirements, even late in development.
- 3 Deliver working software frequently.
- 4 Business people and developers work together daily.
- 5 Build projects around motivated individuals.
- 6 Face-to-face conversation is most efficient.
- 7 Working software is the primary measure of progress.
- 8 Sustainable development (constant pace).
- 9 Continuous attention to technical excellence.
- 10 Simplicity (maximising work *not* done) is essential.
- 11 Self-organising teams generate best architectures.
- 12 Regular reflection and tuning of behaviour.

Iterative vs Incremental



- **Iterative:** Refining the solution through repeated cycles (Moving from vague to concrete).
- **Incremental:** Building the solution piece by piece (Adding features).

Agile Pros and Cons

Advantages

- Flexibility and Adaptability
- Customer Satisfaction
- Improved Quality
- Risk Management
- Higher Productivity
- Enhanced Communication

Disadvantages

- Less Predictability
- Requires high Customer Involvement
- Resource Intensive (Senior talent)
- Scope Creep
- Steep Learning Curve

Project Suitability

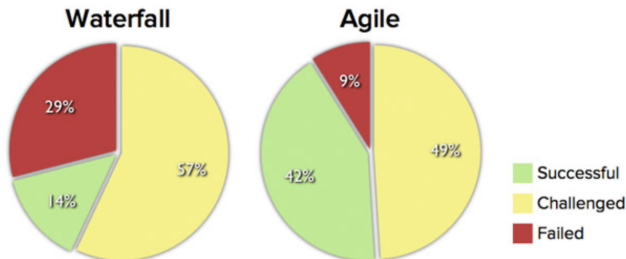
Suited for Agile:

- Projects with uncertain/evolving requirements.
- Rapid delivery requirements.
- Innovative/Creative projects.
- Customer-centric products.

Less Suitable:

- Rigid regulatory requirements.
- Projects where change is costly/dangerous.
- Massive legacy projects difficult to break down.
- Fixed-price, fixed-scope contracts.

Agile Success



Source: The CHAOS Manifesto, The Standish Group, 2012.

Agile projects are roughly 2X more likely to succeed than Waterfall projects, and Waterfall projects are 3X more likely to fail completely.

Agenda

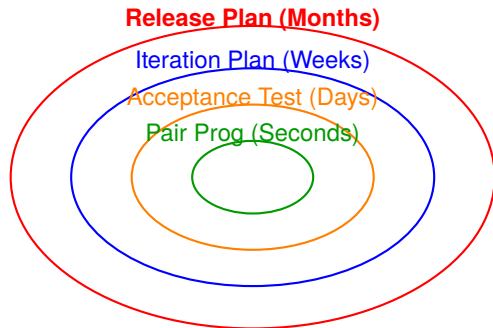
- 1 Agile Overview
- 2 Extreme Programming (XP)
- 3 Scrum
- 4 Agile for Solo Projects
- 5 Next Steps

Extreme Programming (XP)

Introduced in the late 1990s by Kent Beck.

- Focuses on continuous feedback and customer involvement.
- Emphasises technical excellence and good programming practices.
- Taking good practices (like testing and reviewing) to the "extreme".

XP Feedback Loops

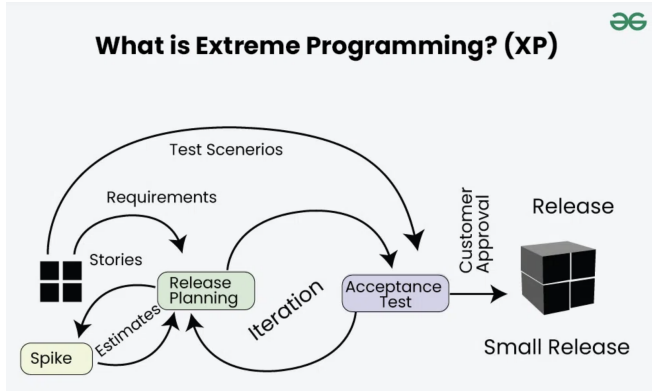


Layers of Feedback in XP

XP Roles

- **Customer:** Defines user stories, provides acceptance criteria, responsible for business value.
- **Developer:** Writes code, tests, and collaborates via pair programming.
- **Tracker:** Tracks progress (velocity), bugs, and issues. Ensures the team is on target.
- **Coach:** Guides the team in XP methodology, facilitates meetings, and helps resolve conflicts.

XP Core Practices



XP Core Practices: User Stories

The Tool

A short, simple description of a feature told from the perspective of the person who desires the new capability.

Standard Format:

"As a **<role>**, I want to **<action>**, so that **<benefit>**."

Example: "As a *registered user*, I want to *view my order history*, so that *I can track my purchases*."

XP Core Practices: Spikes & Refactoring

Spike

- A task aimed at answering a question or gathering information rather than producing shippable code.
- Used when estimates are uncertain due to technical unknowns.
- "Architectural Spike" vs "Risk-based Spike".

Refactoring

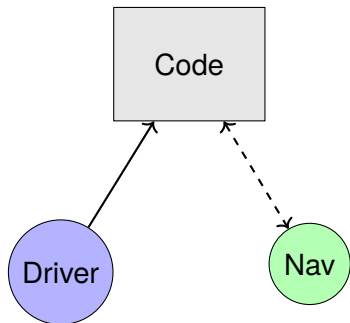
- Restructuring existing code without changing its external behaviour.
- Improves non-functional attributes (readability, reduced complexity).
- Essential in XP to keep the design simple as features are added.

XP Core Practices: Pair Programming

Two programmers working on a single machine.

- **The Driver:** Writes the code. Focuses on the current line of code/tactics.
- **The Navigator:** Reviews the code as it is typed. Focuses on strategy, bugs, and overall design.

Roles switch frequently.



XP Core Practices: Test-Driven Development (TDD)

Definition

A software development process where you write tests before writing the code that implements the functionality.

The TDD Cycle:

Red → Write a failing test.

Green → Write just enough code to pass the test.

Refactor → Clean up the code while keeping tests green.

Agenda

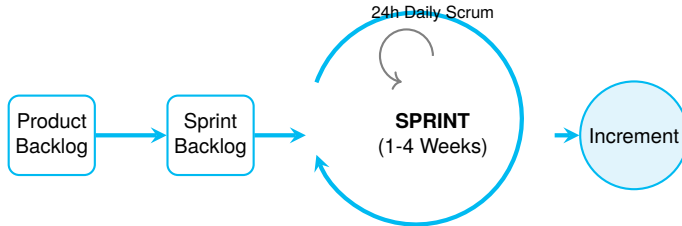
- 1 Agile Overview
- 2 Extreme Programming (XP)
- 3 Scrum**
- 4 Agile for Solo Projects
- 5 Next Steps

Scrum Framework

Scrum is an agile framework that helps teams work together. It encourages teams to learn through experiences, self-organise while working on a problem, and reflect on their wins and losses.

- **Sprints:** Fixed length iterations (usually 1-4 weeks).
- **Artifacts:** Product Backlog, Sprint Backlog, Increment.
- **Roles:** Product Owner, Scrum Master, Development Team.

The Scrum Cycle



Scrum Roles

Product Owner (PO)

Represents stakeholders. Defines project goals, manages and prioritises the Product Backlog. Ensures value delivery.

Scrum Master

Facilitates the Scrum process. Removes impediments (blockers). Protects the team from external interference. Coach and guide.

Development Team

Cross-functional, self-organising group. They do the work (design, code, test) to deliver the Increment.

Scrum Events

- **Sprint Planning:** Team selects items from the Product Backlog to commit to for the Sprint.
- **Daily Scrum (Stand-up):** 15-minute sync. "What did I do yesterday? What will I do today? Blockers?"
- **Sprint Review:** Demo the work to stakeholders. Gather feedback.
- **Sprint Retrospective:** Team reflects on the process. "What went well? What can be improved?"

Scrum Artifacts

- 1 **Product Backlog:** An ordered list of everything needed in the product. Dynamic and prioritized by value.
- 2 **Sprint Backlog:** The subset of items selected for a specific Sprint, plus the plan to deliver them.
- 3 **Increment:** The sum of all backlog items completed during a Sprint + value of previous sprints. Must be "Done" (usable).

XP vs Scrum

XP	Scrum
Iterations typically 1-2 weeks	Sprints typically 2-4 weeks
Changes allowed within iteration if swapped	Scope is fixed during Sprint
Strict engineering practices (TDD, Pair Programming) are mandatory	Focuses on management framework; engineering practices are optional
Feedback via Code/Tests	Feedback via Review/Retro

Agenda

- 1 Agile Overview
- 2 Extreme Programming (XP)
- 3 Scrum
- 4 Agile for Solo Projects**
- 5 Next Steps

Can Agile work for One Person?

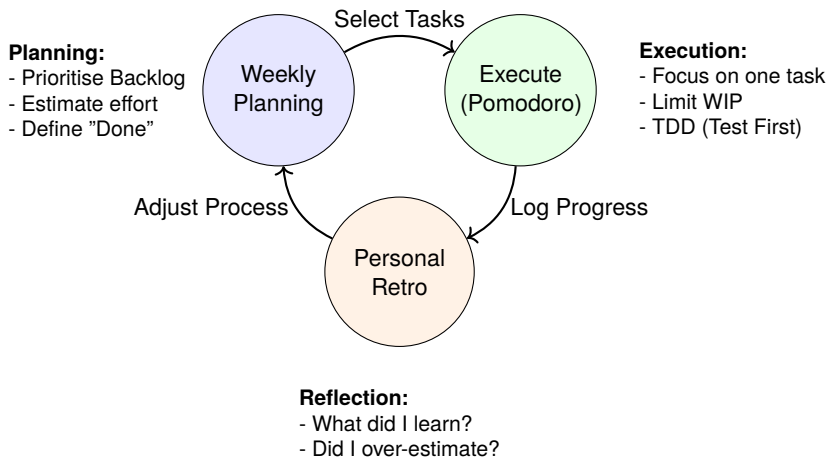
The "Personal Scrum" Concept

Agile is not limited to teams. "Personal Agility" applies the same values to manage individual workload and stress.

The Solo Developer wears three hats:

- 1 **Product Owner:** You decide *what* to build next based on value.
- 2 **Scrum Master:** You ensure you are working effectively and removing distractions.
- 3 **Development Team:** You do the design, coding, and testing.

Adapting the Cycle for Solo Dev



Practical Practices for Students

- **Micro-Sprints:** Instead of 2-4 weeks, run 1-week sprints to get faster feedback on your progress.
- **Daily Self-Standup:** Start your day by writing down:
 - What did I finish yesterday?
 - What will I focus on today?
 - Am I stuck? (If yes, ask a tutor/peer immediately).
- **Personal Kanban:** Use Trello or GitHub Projects.
 - Columns: *Backlog* → *This Week* → *Doing (Max 1)* → *Done*.
- **Definition of Done:** Don't move a card to "Done" until it works and is committed to Git.

Academic & Practitioner Resources for Solo Agile:

- **"Personal Extreme Programming - An Agile Process for Autonomous Developers"** (Research article). Adapts XP practices like TDD and Refactoring for solo use.
- **"Going Agile on your own"** (Blog post). A detailed guide on shifting from "mindless productivity" to effectiveness.
- **"The Personal Scrum Guide"** (Blog post). A framework specifically designed for individual freelancers and students.
- **"Personal Kanban: Mapping Work - Navigating Life"** (Book). Focuses on visualizing work to reduce cognitive load.

Agenda

- 1 Agile Overview
- 2 Extreme Programming (XP)
- 3 Scrum
- 4 Agile for Solo Projects
- 5 Next Steps**

- **Recommended Reading:** Sommerville, *Software Engineering 9th Edition*, Chapter 2 (Software Processes) & Chapter 3 (Agile).
- **Workshop This Week:** Setting up your Scrum project using **Trello**.
- **Next Week:** Software Requirements (Gathering, Analysis, Specification).

Any Questions?

`fdelduchetto@lincoln.ac.uk`